SJMS v4.0 — System Scale Summary

Metric	Count
API Endpoints (GET/POST/PATCH/PUT/DELETE)	**416**
Route Mounts (in index.ts)	**97**
Router Files	**85**
Service Files	**69**
Controller Files	**65**
Test Files	**84**
Tests	**2,242** (0 failures)
n8n Workflow JSON Files	**125**
n8n Workflow Templates (in service)	**62+** (template definitions across 16 categories)
Prisma Database Models	**237**
Prisma Enums	**23**
Database Tables (live)	**212**
Frontend Pages	**114** (44 wired to APIs)

416 API endpoints, **125 n8n workflows**, and **237 data models**.

Database Seeded Data

Entity	Count	Details
Faculties	5	Business & Law, Health & Social Care, Science & Technology, Arts & Education, Creative Industries
Departments	25	5 per faculty (Accounting, Computer Science, Nursing, etc.)
Programmes	60	40 UG (BSc/BA/BEng/LLB), 15 PGT (MSc/MA/MBA), 5 PGR (PhD)
Modules	565	~10 per programme, with credits and HESA codes
Students	500	350 UG, 100 PGT, 50 PGR — all with enrolments and module registrations
Applicants	300	200 UG + 100 PGT, with applications and qualifications
Staff Users	150	80 academic, 40 admin, 20 management, 10 support — with role assignments
Assessments	1,130	2 per module (coursework + exam)
Marks	2,354	Released marks for UG students
Attendance	2,500	5 records per student
Fees	500	£9,250 home UG, £15,000 international, £12,000 PGT
Module Registrations	2,420	Students linked to their modules
Total DB Tables	238	All Prisma models active

All Data Uses FHE Fake Details

- **Addresses**: "XX Future Horizons Lane/Road/Drive, Future Horizons City, FH1-9 XXX"
- **Emails**: firstname.lastname@fhe.ac.uk
- **Phones**: 020 7946 XXXX (UK Ofcom test range)
- **Institution**: Future Horizons University, UKPRN: 10099999

Database: 238 tables, seeded with 60 programmes, 500 students, 300 applicants, 150 staff

HESA Data Architecture in SJMS v4

HESA data is organised across **4 layers** in the system:

Layer 1: HESA Fields Embedded in Core Models

HESA-reportable fields are stored directly on the entities they describe — not in separate tables:

Core Model	HESA Fields	Purpose
Student	husid (13-digit ID), numhus, feeEligibility (FEEELIG), majorFeeSource (MSTUFEE), domicile (DOMICILE), hesaSexId (SEXID), qualificationOnEntry (QUALENT3)	Student demographic return
Enrolment	hesaEngagementId, yearOfProgramme, fundingCompletion (FUNDCOMP), typeOfYear (TYPEYR), instancePeriod	Student course session data
Programme	hesaCourseId (COURSEID), courseAim (COURSEAIM), hecosCode1/hecosCode2 (HECoS classification), hecosPercentage1/2, ttcid	Course data
Module	hesaModuleId (MODID), creditLevel (LEVLPTS), hecosCode	Module instance data
Student (encrypted)	ethnicity (ETHNIC), disability (DISABLE)	Protected characteristics
StaffRecord	hesaStaffId, contractLevel (A0-F1), socCode (SOC2020), fte/fpe	Staff data futures

Layer 2: HESA Reference Data Tables (7 models)

These store the HESA coding frames — the allowed values for each HESA field:

Model	Location in Schema	Purpose
HesaCodingFrame	Line 2730	All HESA codesets (ethnicity 10-90, disability 00-99, etc.)
HesaCostCentre	Line 4158	Institution cost centre mappings
DepartmentCostCentre	Line 4176	Department-to-cost-centre mapping
HesaHecosCode	Line 4194	HECoS subject classification codes
HesaQualificationAim	Line 4217	Qualification aim codes (H00, M00, D00, etc.)
HesaReasonForEndingLookup	Line 4232	Standard withdrawal/completion reason codes
HesaStaffContractLevel	Line 4243	Staff contract level classifications
HesaSocCode	Line 4255	SOC2020 occupation codes

Layer 3: HESA Quality & Validation (5 models)

Data quality enforcement and validation rules:

Model	Purpose
HesaQualityRule	24+ validation rules (HUSID format, coding frame checks, cross-entity)
DataQualityRule	Generic validation rules with hesaMandatory flag
DataQualityAlert	Individual validation failures
DataQualityBatchRun	Batch validation execution tracking
DataChangeLog	Field-level change audit

Layer 4: HESA Returns & Submissions (3 models)

Managing the actual statutory return lifecycle:

Model	Purpose
HesaReturn	Return record (type, status, submission tracking, HESA reference number)
HesaReturnError	Credibility errors with resolution tracking
StatutoryReturn	Generic statutory returns (WP2, ACCESS_AGREEMENT, etc.)

API Endpoints (3 modules, 20+ endpoints)

Module	Path	Key Endpoints
HESA Quality	/api/hesa/*	POST /validate, GET /dashboard, POST /husid/generate, POST /husid/batch, GET /mappings/:entity, POST /export, GET /rules
HESA Returns	/api/hesa-returns/*	GET /, POST /, PUT /:id/status, POST /:id/validate, GET /:id/errors, POST /:id/submit, GET /heses/snapshot, GET /data-futures/student-extract
Data Futures	/api/integrations/hesa-data-futures/*	Validation with 50 synthetic test records, credibility error generation

HUSID Generation Algorithm

Format: 13-digit number with modulus-11 check digit

Weights: [2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 2, 3]

Check: 11 - (Σ(digit × weight) mod 11), if 10 or 11 → use 0

Key Files

prisma/schema.prisma — All 15 HESA models

server/src/api/compliance/hesa-quality.service.ts — 24 validation rules + HUSID algorithm

server/src/api/compliance/hesa-quality.controller.ts — Quality API handlers

server/src/api/hesa-returns/hesa-returns.router.ts — Returns lifecycle API (719 lines)

server/src/api/integrations/hesa-data-futures.router.ts — Data Futures validation

server/src/utils/hesa-xml-generator.ts — XML export generation

SJMS 4.0 Data Context Map

```
┌──────────────────────────────────────────────────────────────────┐
│  STUDENT IDENTITY (Red Domain — highest coupling)                 │
│  Student → User (1:1), StudentStatusHistory                       │
│   ┌──────────────────────────────────────────┐                    │
│   │ 21 files with direct Prisma operations   │                    │
│   │ 15+ relations to other entities          │                    │
│   └──────────────────────────────────────────┘                    │
├──────────────────────────────────────────────────────────────────┤
│  ENROLMENT LIFECYCLE (Red Domain)                                 │
│  Enrolment → Student, Programme, AcademicYear                     │
│  ModuleRegistration → Enrolment, Module, AcademicYear             │
│   ┌──────────────────────────────────────────┐                    │
│   │ 12 files with direct Prisma operations   │                    │
│   │ Unique constraint: student+prog+year     │                    │
│   └──────────────────────────────────────────┘                    │
├──────────────────────────────────────────────────────────────────┤
│  ASSESSMENT & MARKS (Amber Domain)                                │
│  Mark → Assessment, ModuleRegistration, Student                   │
│  Assessment → Module, AcademicYear                                │
│   ┌──────────────────────────────────────────┐                    │
│   │ 6 files with direct Prisma operations    │                    │
│   │ onDelete: Restrict (protects marks)      │                    │
│   │ 7-stage pipeline: DRAFT → RELEASED       │                    │
│   └──────────────────────────────────────────┘                    │
├──────────────────────────────────────────────────────────────────┤
│  FINANCE (Amber Domain)                                           │
│  Fee → Student, Enrolment, Application                            │
│  Invoice → Fee | Payment → Fee, Invoice                           │
│   ┌──────────────────────────────────────────┐                    │
│   │ 5 files with direct Prisma operations    │                    │
│   │ Decimal(10,2) precision                  │                    │
│   │ SLC integration dependency               │                    │
│   └──────────────────────────────────────────┘                    │
├──────────────────────────────────────────────────────────────────┤
│  COMPLIANCE & AUDIT (Green Domain — append-only)                  │
│  AuditLog → User | DataChangeLog (append-only)                    │
│  HesaReturn → HesaReturnError                                     │
│   ┌──────────────────────────────────────────┐                    │
│   │ 4 files with direct Prisma operations    │                    │
│   │ Append-only: never UPDATE/DELETE          │                    │
│   │ hesaImpact flag on DataChangeLog          │                    │
│   └──────────────────────────────────────────┘                    │
└──────────────────────────────────────────────────────────────────┘
```

Schema Dependency Audit — SJMS v4.0

1. Objective

Inventory every consumer of the 8 core entities (Student, Enrolment, ModuleRegistration, Mark, Fee, HesaReturn, AuditLog, and closely related models) to produce a safe migration order before any schema changes. This document is the gate check — no schema edits proceed without it.

Student (21 files, 59 Prisma operations)

File	Context	Ops	R/W
students.service.ts	Core CRUD	findUnique(4), findMany(2), findFirst(1), create(1), update(2), count(2)	R/W
students.controller.ts	Request handling	findFirst(1)	R
dashboard.service.ts	KPI dashboard	count(3), findUnique(1)	R
analytics.service.ts	Reporting	findMany(5), count(2)	R
attendance.service.ts	Attendance tracking	findUnique(4)	R
support.service.ts	Support tickets	findUnique(2)	R
gdpr.service.ts	Data erasure	findUnique(1), update(1)	R/W
ukvi-compliance.service.ts	Visa checks	findUnique(2)	R
applicant-transform.service.ts	Applicant→Student	findUnique(4), findMany(1), create(1)	R/W
online-enrolment.service.ts	Enrolment flow	findUnique(1)	R
student-portal.service.ts	Student portal (14 functions)	findFirst(14), findMany(6)	R/W
export-adapter.router.ts	CSV/JSON export	findMany(2), count(1)	R
dynamics365-adapter.router.ts	CRM sync	findMany(1), findFirst(1)	R
sharepoint-adapter.router.ts	SharePoint sync	findMany(1), findFirst(1)	R
scripts/seed.ts	Data seeding	deleteMany(), create(10000)	W
+ 6 test files	Testing	Various	R/W

Enrolment (12 files, 25 Prisma operations)

File	Context	Ops	R/W
dashboard.service.ts	KPI counts	count(7)	R
online-enrolment.service.ts	Create enrolment	create(1), findFirst(1)	R/W
applicant-transform.service.ts	Conversion	findUnique(1), findFirst(1), create(1)	R/W
enrolment.router.ts	Status updates	update(1)	W
student-portal.service.ts	Student view	findFirst(3)	R
export-adapter.router.ts	Export stats	count(1)	R
ukvi-compliance.service.ts	Visa compliance	findFirst(1), findMany(1)	R
scripts/seed.ts	Seeding	deleteMany(), create(500)	W
+ 2 test files	Testing	count, groupBy	R

ModuleRegistration (8 files, 12 Prisma operations)

File	Context	Ops	R/W
student-portal.service.ts	Student modules	findMany(1), create(per module selection)	R/W
attendance.service.ts	Validate registration	findUnique(2)	R
modules.service.ts	Student list per module	findMany(1)	R
dashboard.service.ts	Module metrics	findMany(1), count(1)	R
applicant-transform.service.ts	Enrolment completion	create(1)	W
scripts/seed.ts	Seeding	deleteMany(), create(~2500)	W
+ 2 test files	Testing	findMany, findUnique	R

Mark (6 files, 15 Prisma operations)

File	Context	Ops	R/W
assessment-workflow.service.ts	Mark entry + release	create(TXN), findMany(3), updateMany(1)	R/W
exam-board.service.ts	Board analysis	findMany(1)	R
student-portal.service.ts	Student marks view	findMany(2)	R
scripts/seed.ts	Seeding	deleteMany(), create(~3500)	W
+ 2 test files	Testing	Various	R

Fee (5 files, 10 Prisma operations)

File	Context	Ops	R/W
dashboard.service.ts	Overdue count	count(1)	R
student-portal.service.ts	Student finance	findMany(2)	R
analytics.service.ts	Revenue reporting	findMany(1)	R
finance.router.ts	Fee CRUD	findMany, create, update	R/W
scripts/seed.ts	Seeding	deleteMany(), create(500)	W

HesaReturn (3 files)

File	Context	Ops	R/W
hesa-returns.router.ts	Return lifecycle (719 lines)	Mock/demo responses	R (mock)
hesa-quality.service.ts	Quality validation	Student queries for validation	R
hesa-data-futures.router.ts	Data Futures extract	Student/Enrolment queries	R

Note: HesaReturn CRUD is not yet Prisma-backed — still mock data. Low migration risk.

AuditLog (4 files, append-only)

File	Context	Ops	R/W
audit.service.ts	Central audit	create(1), findMany(1)	R/W
middleware/audit.ts	Auto-logging	Calls audit.service.log()	W
dashboard.service.ts	Recent activity	findMany(1)	R
audit.test.ts	Testing	Various	R

Note: Append-only pattern — AuditLog records are never updated or deleted.

3.3 External System Dependencies

External System	Entities Consumed	Direction	Coupling
n8n (74 workflows)	Student, Enrolment, Mark, Fee, Attendance	Webhook payloads → n8n	Loose (HTTP)
SLC	Student, Fee, Enrolment	Bidirectional	Medium (API adapter)
UCAS	Applicant, Application	Inbound	Low (mock adapter)
HESA	Student, Enrolment, Module, Mark	Outbound extract	High (field mapping)
Dynamics 365	Student, Enrolment, Fee	Outbound push	Low (new, no live sync)
SharePoint	Student, Enrolment	Outbound push	Low (new, no live sync)
Moodle	Student, Module, ModuleRegistration	Bidirectional sync	Medium

4. Target-State Design

4.1 Entity-Relationship Diagram

```
Person (central identity)
├── PersonName[]          (LEGAL, PREFERRED, PREVIOUS — effective-dated)
├── ContactMethod[]       (EMAIL, PHONE, MOBILE — effective-dated)
├── AddressUsage[]        → Address (HOME, CORRESPONDENCE, TERM_TIME)
├── Nationality[]         (ISO country codes — effective-dated)
├── Citizenship[]         (CITIZEN, SETTLED, PRE_SETTLED — effective-dated)
├── ResidencyStatus[]     (HOME, EU, INTERNATIONAL + HESA domicile/feeElig)
├── EmergencyContact[]    (prioritized, effective-dated)
├── IdentityDocument[]    (PASSPORT, BRP, etc. — encrypted)
├── SensitiveAttribute[]  (ETHNICITY, DISABILITY, etc. — encrypted, Art.9)
├── ConsentRecord[]       (MARKETING, HESA_RETURN, etc.)
└── LawfulBasisRecord[]   (Art.6/Art.9 basis — effective-dated)

Student.personId? ──→ Person  (backward-compatible, optional)
Applicant.personId? ──→ Person  (enables applicant→student identity continuity)
Staff.personId? ───→ Person  (unifies staff identity)
StaffRecord.personId? → Person
```

4.3 Cross-Entity Identity Deduplication

When the same physical person exists as both Applicant and Student (sharing the same email), the migration script creates **one Person** and links both:

```
Person (id: "abc")
├── Student (personId: "abc")   — enrolled student
└── Applicant (personId: "abc") — original application record
```

4.4 New Enums (10)

Enum	Values	Used By
`NameType`	LEGAL, PREFERRED, PREVIOUS, ALIAS	PersonName
`ContactType`	EMAIL, PHONE, MOBILE, ALTERNATIVE_EMAIL	ContactMethod
`AddressUsageType`	HOME, CORRESPONDENCE, TERM_TIME, NEXT_OF_KIN, PERMANENT	AddressUsage
`CitizenshipType`	CITIZEN, PERMANENT_RESIDENT, SETTLED, PRE_SETTLED, INDEFINITE_LEAVE	Citizenship
`ResidencyType`	HOME, EU, INTERNATIONAL, ISLANDS, REFUGEE, OTHER	ResidencyStatus
`IdentityDocType`	PASSPORT, DRIVING_LICENCE, BRP, NATIONAL_ID, BIRTH_CERTIFICATE	IdentityDocument
`SensitiveAttributeType`	ETHNICITY, DISABILITY, RELIGION, SEXUAL_ORIENTATION, GENDER_IDENTITY	SensitiveAttribute
`ConsentPurpose`	MARKETING, RESEARCH, HESA_RETURN, DISABILITY_SUPPORT, ALUMNI, GRADUATE_OUTCOMES	ConsentRecord
`ConsentStatus`	GRANTED, WITHDRAWN, EXPIRED	ConsentRecord
`LawfulBasisType`	CONSENT, CONTRACT, LEGAL_OBLIGATION, VITAL_INTERESTS, PUBLIC_TASK, LEGITIMATE_INTERESTS	LawfulBasisRecord
`Article9Condition`	EXPLICIT_CONSENT, EMPLOYMENT, VITAL_INTERESTS, PUBLIC_INTEREST, HEALTH, RESEARCH	LawfulBasisRecord

Entity Hierarchy

```
Student
└── StudyAim[]  (one per qualification goal — BSc CS, then later MSc DS)
    └── StudentInstance[]  (one per attempt at the study aim)
        ├── InstancePeriod[]  (one per academic period — Year 1, Year 2, etc.)
        |   └── EnrolmentOccurrence[]  (registration events per period)
        ├── ModeOfStudyHistory[]  (FT→PT transitions)
        ├── InterruptionEvent[]  (interruption records)
        ├── TransferEvent[]  (programme/institution transfers)
        ├── CompletionEvent?  (completion with classification)
        └── WithdrawalEvent?  (withdrawal with reason)

ProgrammeOccurrence  (programme delivery per academic year — standalone)
```

New Enums (3)

```
enum InstanceStatus {
  INST_ACTIVE
  INST_DORMANT
  INST_INTERRUPTED
  INST_COMPLETED
  INST_WITHDRAWN
  INST_TRANSFERRED_OUT
  INST_TRANSFERRED_IN
}

enum LifecycleEventType {
  EVT_INTERRUPTION
  EVT_RETURN_FROM_INTERRUPTION
  EVT_WITHDRAWAL
  EVT_COMPLETION
  EVT_TRANSFER_OUT
  EVT_TRANSFER_IN
  EVT_MODE_CHANGE
  EVT_PROGRAMME_CHANGE
}

enum TransferType {
  INTERNAL_PROGRAMME
  INTERNAL_CAMPUS
  EXTERNAL_INSTITUTION
  CLEARING_TRANSFER
}
```

New Models (10)

1. StudyAim — A student's overarching qualification goal. Maps to HESA COURSEID/COURSEAIM.

```
model StudyAim {
  id              String    @id @default(cuid())
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt
  createdBy       String
  updatedBy       String
  deletedAt       DateTime?
  studentId       String
  programmeId     String
  hesaCourseId    String?   /// HESA COURSEID
  courseAim        String?   /// HESA COURSEAIM (H00, M00, D00, etc.)
  startDate       DateTime
  expectedEndDate DateTime?
  actualEndDate   DateTime?
  isActive        Boolean   @default(true)

  student    Student    @relation(fields: [studentId], references: [id])
  programme  Programme  @relation(fields: [programmeId], references: [id])
  instances  StudentInstance[]

  @@index([studentId])
  @@index([programmeId])
  @@index([studentId, isActive])
  @@map("study_aims")
}
```

2. StudentInstance — A specific attempt/registration on a study aim.

```
model StudentInstance {
  id                  String         @id @default(cuid())
  createdAt           DateTime       @default(now())
  updatedAt           DateTime       @updatedAt
  createdBy           String
  updatedBy           String
  deletedAt           DateTime?
  studyAimId          String
  enrolmentId         String?        /// Bridge to existing Enrolment (backward cc
  instanceNumber      Int            @default(1) /// 1st attempt, 2nd attempt, etc
  status              InstanceStatus @default(INST_ACTIVE)
  hesaEngagementId    String?        /// HESA engagement identifier
  startDate           DateTime
  expectedEndDate     DateTime?
  actualEndDate       DateTime?
  modeOfStudy         ModeOfStudy
  studyIntensity      Float          @default(1.0)
  fundingSource       String?
  sponsorId           String?

  studyAim             StudyAim              @relation(fields: [studyAimId], refer
  enrolment            Enrolment?            @relation(fields: [enrolmentId], refe
  instancePeriods      InstancePeriod[]
  modeOfStudyHistory   ModeOfStudyHistory[]
  interruptionEvents   InterruptionEvent[]
  transferEvents       TransferEvent[]
  completionEvent      CompletionEvent?
  withdrawalEvent      WithdrawalEvent?

  @@index([studyAimId])
  @@index([enrolmentId])
  @@index([status])
  @@map("student_instances")
}
```

3. **ProgrammeOccurrence** — A specific delivery of a programme in an academic year.

```
model ProgrammeOccurrence {
  id              String    @id @default(cuid())
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt
  createdBy       String
  updatedBy       String
  deletedAt       DateTime?
  programmeId     String
  academicYearId  String
  intakeMonth     String?   /// e.g., "September", "January"
  maxPlaces       Int?
  currentPlaces   Int       @default(0)
  isActive        Boolean   @default(true)

  programme       Programme    @relation(fields: [programmeId], references: [id])
  academicYear    AcademicYear @relation(fields: [academicYearId], references: [id])
  instancePeriods InstancePeriod[]

  @@unique([programmeId, academicYearId, intakeMonth])
  @@index([programmeId])
  @@index([academicYearId])
  @@map("programme_occurrences")
}
```

4. **InstancePeriod** — A period within a student instance (Year 1, Year 2, placement year).

```
model InstancePeriod {
  id                    String    @id @default(cuid())
  createdAt             DateTime  @default(now())
  updatedAt             DateTime  @updatedAt
  createdBy             String
  updatedBy             String
  deletedAt             DateTime?
  studentInstanceId     String
  programmeOccurrenceId String?
  periodNumber          Int       /// 1, 2, 3, etc.
  hesaTypeOfYear        String?   /// HESA TYPEYR (1-8)
  hesaFundComp          String?   /// HESA FUNDCOMP
  hesaModeStat          String?   /// HESA MODESTAT
  startDate             DateTime
  endDate               DateTime?
  studyIntensity        Float     @default(1.0)
  creditsAttempted      Int?
  creditsAchieved       Int?

  studentInstance     StudentInstance      @relation(fields: [studentInstanceId], re
  programmeOccurrence ProgrammeOccurrence? @relation(fields: [programmeOccurrenceId]
  enrolmentOccurrences EnrolmentOccurrence[]

  @@unique([studentInstanceId, periodNumber])
  @@index([studentInstanceId])
  @@index([programmeOccurrenceId])
  @@map("instance_periods")
```

5. EnrolmentOccurrence — A specific registration event on a period.

```
model EnrolmentOccurrence {
  id                 String    @id @default(cuid())
  createdAt          DateTime  @default(now())
  updatedAt          DateTime  @updatedAt
  createdBy          String
  updatedBy          String
  deletedAt          DateTime?
  instancePeriodId   String
  registrationDate   DateTime
  confirmationDate   DateTime?
  modeOfStudy        ModeOfStudy
  studyIntensity     Float     @default(1.0)
  feeStatus          String?   /// HESA FEEELIG
  isConfirmed        Boolean   @default(false)

  instancePeriod InstancePeriod @relation(fields: [instancePeriodId], references: [i

  @@index([instancePeriodId])
  @@map("enrolment_occurrences")
}
```

6. ModeOfStudyHistory — Tracks mode changes with effective dating.

```
model ModeOfStudyHistory {
  id                 String      @id @default(cuid())
  createdAt          DateTime    @default(now())
  updatedAt          DateTime    @updatedAt
  createdBy          String
  updatedBy          String
  deletedAt          DateTime?
  studentInstanceId  String
  previousMode       ModeOfStudy?
  newMode            ModeOfStudy
  effectiveDate      DateTime
  reasonCode         String?
  reasonDetail       String?
  approvedBy         String?
  approvedDate       DateTime?

  studentInstance StudentInstance @relation(fields: [studentInstanceId], references:

  @@index([studentInstanceId])
  @@index([effectiveDate])
  @@map("mode_of_study_history")
}
```

7. **InterruptionEvent** — Records an interruption with full traceability.

```
model InterruptionEvent {
  id                  String    @id @default(cuid())
  createdAt           DateTime  @default(now())
  updatedAt           DateTime  @updatedAt
  createdBy           String
  updatedBy           String
  deletedAt           DateTime?
  studentInstanceId   String
  hesaNotActiveCode   String?   /// HESA NOTACT code
  reasonCode          String?   /// HESA RSNEND code
  reasonDetail        String?
  interruptionDate    DateTime
  expectedReturnDate  DateTime?
  actualReturnDate    DateTime?
  approvedBy          String?
  approvedDate        DateTime?

  studentInstance StudentInstance @relation(fields: [studentInstanceId], references:

  @@index([studentInstanceId])
  @@index([interruptionDate])
  @@map("interruption_events")
}
```

8. **TransferEvent** — Records programme or institution transfers.

```
model TransferEvent {
  id                    String       @id @default(cuid())
  createdAt             DateTime     @default(now())
  updatedAt             DateTime     @updatedAt
  createdBy             String
  updatedBy             String
  deletedAt             DateTime?
  studentInstanceId     String
  transferType          TransferType
  fromProgrammeId       String?
  toProgrammeId         String?
  fromInstitution       String?      /// UKPRN of source institution
  toInstitution         String?      /// UKPRN of destination institution
  transferDate          DateTime
  creditsTransferred    Int?
  reasonCode            String?
  approvedBy            String?
  approvedDate          DateTime?

  studentInstance StudentInstance @relation(fields: [studentInstanceId], references:

  @@index([studentInstanceId])
  @@index([transferDate])
  @@map("transfer_events")
}
```

9. CompletionEvent — Records completion with classification.

```
model CompletionEvent {
  id                  String               @id @default(cuid())
  createdAt           DateTime             @default(now())
  updatedAt           DateTime             @updatedAt
  createdBy           String
  updatedBy           String
  deletedAt           DateTime?
  studentInstanceId   String               @unique /// One completion per instance
  completionDate      DateTime
  classification      DegreeClassification?
  totalCredits        Int?
  gpa                 Float?
  awardTitle          String?
  hesaRsnEnd          String?              /// HESA RSNEND code
  boardApprovedDate   DateTime?
  certificateIssued   Boolean              @default(false)

  studentInstance StudentInstance @relation(fields: [studentInstanceId], references:

  @@index([completionDate])
  @@map("completion_events")
}
```

10. WithdrawalEvent — Records withdrawal with HESA reason codes.

```
model WithdrawalEvent {
  id                  String    @id @default(cuid())
  createdAt           DateTime  @default(now())
  updatedAt           DateTime  @updatedAt
  createdBy           String
  updatedBy           String
  deletedAt           DateTime?
  studentInstanceId   String    @unique /// One withdrawal per instance
  withdrawalDate      DateTime
  lastAttendanceDate  DateTime?
  hesaRsnEnd          String?   /// HESA RSNEND code (01-11, 98, 99)
  reasonDetail        String?
  isVoluntary         Boolean   @default(true)
  refundEligible      Boolean   @default(false)
  approvedBy          String?
  approvedDate        DateTime?

  studentInstance StudentInstance @relation(fields: [studentInstanceId], references:

  @@index([withdrawalDate])
  @@map("withdrawal_events")
}
```

Bridge: Existing Enrolment → StudentInstance

Add optional FK to existing Enrolment (and add reverse relation from Enrolment):

```
// On Enrolment model:
  studentInstances StudentInstance[]

// On Student model:
  studyAims StudyAim[]
```

And on Programme:

```
  studyAims            StudyAim[]
  programmeOccurrences ProgrammeOccurrence[]
```

On AcademicYear:

```
  programmeOccurrences ProgrammeOccurrence[]
```

HESA Data Structures

```
Operational Domain (live data)          HESA Reporting Domain (snapshot data)
──────────────────────────────          ────────────────────────────────────

Student ──────extract──────────────────→ HesaLearner
Enrolment / StudentInstance ──extract──→ HesaEngagement
InstancePeriod ──────────────extract──→ HesaStudentCourseSession
                                        HesaSessionYear
Programme ──────────────────extract──→ HesaQualification
                                        HesaQualificationSubject
Module ──────────────────────extract──→ HesaModule
                                        HesaModuleSubject

                                        HesaReturnSnapshot (immutable per submission
                                        HesaSubmissionRun (workflow tracking)
                                        HesaValidationResult (per-record errors)
                                        HesaCodingFrameVersion (version tracking)
```

New Enums (2)

```
enum HesaSnapshotStatus {
  HSNAP_DRAFT
  HSNAP_EXTRACTED
  HSNAP_VALIDATED
  HSNAP_APPROVED
  HSNAP_SUBMITTED
  HSNAP_ACCEPTED
  HSNAP_REJECTED
}

enum HesaValidationSeverity {
  HVAL_ERROR
  HVAL_WARNING
  HVAL_ADVISORY
  HVAL_INFO
}
```

1. HesaLearner

Field	Type	Source	HESA Field
snapshotId	FK → HesaReturnSnapshot	—	—
studentId	FK → Student	Student.id	—
husid	String	Student.husid	HUSID
numhus	String?	Student.numhus	NUMHUS
ownStu	String	Student.studentNumber	OWNSTU
surname	String	Student.lastName	SURNAME
fnames	String	Student.firstName	FNAMES

Field	Type	Source	HESA Field
birthDate	DateTime	Student.dateOfBirth	BIRTHDTE
sexId	String	Student.hesaSexId	SEXID
ethnicId	String?	Student.ethnicity	ETHNIC
disabilityCode	String?	Student.disability	DISABLE
nationCode	String?	Student.nationalityCode	NATION
domicile	String?	Student.domicile	DOMICILE
postcode	String?	Student.postcode	POSTCODE
qualEnt3	String?	Student.qualificationOnEntry	QUALENT3
feeElig	String?	Student.feeEligibility	FEEELIG

2. HesaEngagement .

Field	Source	HESA Field
snapshotId	FK	—
learnerId	FK → HesaLearner	—
engagementId	Enrolment.hesaEngagementId	ENGID
courseId	Programme.hesaCourseId	COURSEID
comDate	Enrolment.enrolmentDate	COMDATE
endDate	Enrolment.actualEndDate	ENDDATE
rsnEnd	WithdrawalEvent/CompletionEvent	RSNEND
modeOfStudy	Enrolment.modeOfStudy (mapped)	MODE
studyLoad	Enrolment.studyIntensity × 100	STULOAD
fundComp	Enrolment.fundingCompletion	FUNDCOMP
typeOfYear	Enrolment.typeOfYear	TYPEYR
yearPrg	Enrolment.yearOfProgramme	YEARPRG
feeRegime	Student.majorFeeSource	MSTUFEE

3. HesaStudentCourseSession — Period-level data for a student on a course.

4. HesaModule — Module as reported to HESA.

Field	Source	HESA Field
snapshotId	FK	—
hesaModuleId	Module.hesaModuleId	MODID
moduleTitle	Module.name	MODTITLE
credits	Module.credits	CRDTPTS
creditLevel	Module.creditLevel	LEVLPTS
hecosCode	Module.hecosCode	MODSBJ

5. HesaModuleSubject — HECoS subject classification splits for a module.

6. HesaQualification — Programme/qualification as reported.

Field	Source	HESA Field
snapshotId	FK	—
courseId	Programme.hesaCourseId	COURSEID
courseTitle	Programme.name	CTITLE
courseAim	Programme.courseAim	COURSEAIM
fheqLevel	Programme.fheqLevel	FHEQLEV

7. HesaQualificationSubject — HECoS subject splits for a qualification.

Field	Source	HESA Field
qualificationId	FK → HesaQualification	—
hecosCode	Programme.hecosCode1/2	SBJCA
percentage	Programme.hecosPercentage1/2	SBJPCNT

8. HesaSessionYear — Academic session/year reference.

9. HesaReturnSnapshot — Immutable point-in-time snapshot of all data for a return.

Field	Type	Purpose
returnId	FK → HesaReturn	Links to return metadata
snapshotDate	DateTime	When snapshot was taken
status	HesaSnapshotStatus	Workflow status
codingFrameVersion	String	Which coding frame version was used
totalLearners	Int	Count of learner records
totalEngagements	Int	Count of engagement records
totalModules	Int	Count of module records
xmlPayload	String?	Generated XML (immutable once submitted)
checksum	String?	SHA-256 of xmlPayload for integrity

10. HesaSubmissionRun — Tracks each attempt to submit a return.

Field	Type	Purpose
snapshotId	FK → HesaReturnSnapshot	Which snapshot was submitted
submittedAt	DateTime	When submitted
submittedBy	String	Who submitted
hesaResponse	String?	Raw HESA response
hesaReferenceNo	String?	HESA-assigned reference
accepted	Boolean?	Whether HESA accepted

11. HesaValidationResult — Per-record validation results.

Field	Type	Purpose
snapshotId	FK → HesaReturnSnapshot	—
entityType	String	Learner, Engagement, Module, etc.
entityId	String	ID of the snapshot entity
fieldName	String?	Which field failed
ruleCode	String	HESA credibility rule code
severity	HesaValidationSeverity	ERROR/WARNING/ADVISORY
message	String	Human-readable error
resolvedAt	DateTime?	When fixed

12. HesaCodingFrameVersion — Tracks which version of each coding frame was active.

Field	Type	Purpose
codingType	String	ETHNIC, DISABLE, MODE, etc.
version	String	e.g., "2025-26"
effectiveFrom	DateTime	When this version became active
effectiveTo	DateTime?	null = current
totalCodes	Int	Number of codes in this version
changesSummary	String?	What changed from previous version

HESA Field	Entity
UCASPERID	HesaLearner
PARED	HesaLearner
SSN	HesaEngagement
SPECFEE	HesaEngagement
GROSSFEE / NETFEE	HesaEngagement
INITIATIVES	HesaEngagement
MODOUT	HesaModule
MODSTAT	HesaModule

Role Hierarchy (27+ roles — Keycloak realm: fhe)

```
super\\\_admin
 ├── system\\\_admin
 ├── registrar
 │   ├── senior\\\_registry\\\_officer
 │   ├── registry\\\_officer
 │   ├── admissions\\\_manager
 │   │   ├── admissions\\\_officer
 │   │   └── admissions\\\_tutor
 │   ├── assessment\\\_officer
 │   ├── progression\\\_officer
 │   └── graduation\\\_officer
 ├── finance\\\_director
 │   ├── finance\\\_manager
 │   └── finance\\\_officer
 ├── quality\\\_director
 │   ├── quality\\\_officer
 │   └── compliance\\\_officer
 ├── dean
 │   ├── associate\\\_dean
 │   ├── head\\\_of\\\_department
 │   │   ├── programme\\\_leader
 │   │   └── module\\\_leader
 │   └── academic\\\_staff
 │       ├── lecturer
 │       ├── senior\\\_lecturer
 │       └── professor
 ├── student\\\_support\\\_manager
```

│ ├── student_support_officer

│ ├── personal_tutor

│ ├── disability_advisor

│ └── wellbeing_officer

├── international_officer

├── accommodation_officer

├── student (authenticated — own data only)

├── applicant (authenticated — own application only)

└── public (unauthenticated — prospectus, course search)